

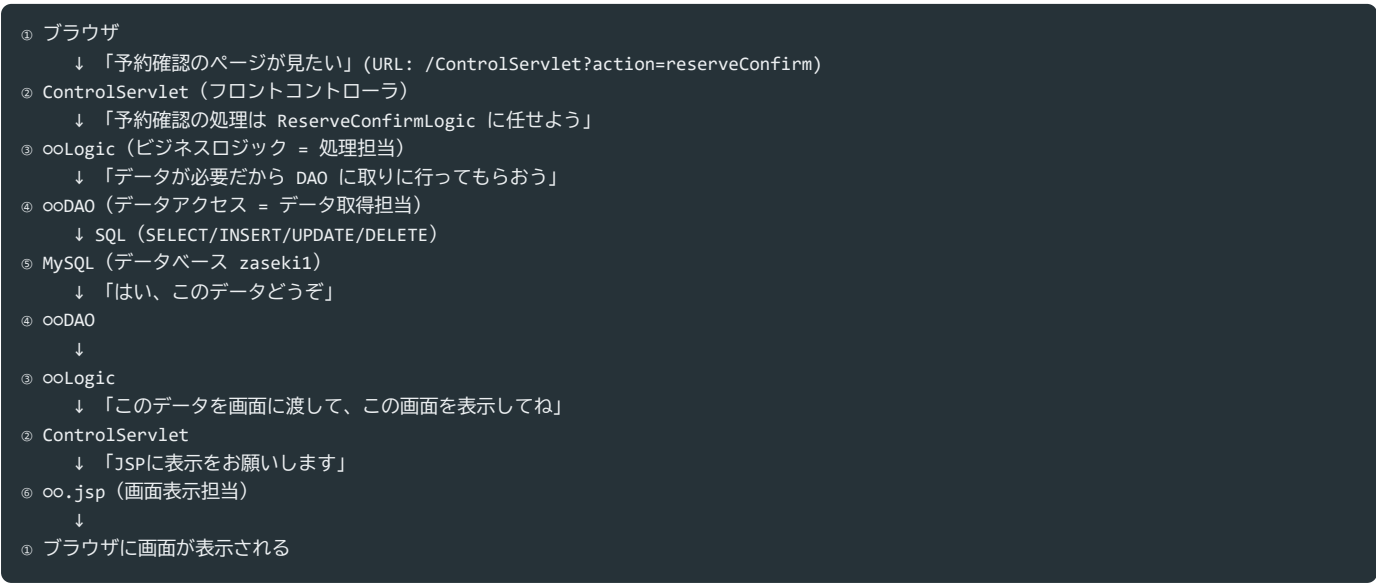
座・Navi プログラム解説書（初学者向け・最新版）

このドキュメントは「座・Navi (Za-Navi)」のプログラムが、それぞれ何をしているのか・どこをどう直すと何が変わるのかをまとめたものです。チームメンバーが「自分はどのファイルを見ればいいのか」「ここを直したら何が起きるか」を理解する手助けになることを目指しています。

★ **このバージョンの更新点:** データの保存先が「Javaコード内のモックデータ (Arrays.asList)」から **MySQL (データベース)** に変わりました。全体の流れ・DAOの説明・Q&Aを最新のコードに合わせて更新し、画面一覧やデータベース構成など「表で全体を見渡せる」資料を追加しています。

1. 全体の流れ（MVCモデル2 + データベース）

このアプリは「**MVCモデル2**」という設計で作られています。難しく聞こえますが、やっていることはシンプルです。



各層の役割を表でみる

層	主なファイル	役割	持ってい知識
① ブラウザ	(HTML/CSS/JS)	画面を表示し、リンク・フォームでリクエストを送る	画面のことだけ
② コントローラ	control/ControlServlet.java	URLの <code>action</code> を見て、担当のLogicに振り分ける	「どのactionをどのLogicに渡すか」だけ
③ ロジック	model/ooLogic.java	画面ごとの処理担当。入力チェック・業務ルール判断	業務ルール (DBの中身は知らない)
④ DAO	dao/ooDAO.java	SQLを実行してデータベースとやりとりする	SQL・テーブル構造のことだけ
⑤ DB	MySQL zaseki1	データの保存場所	-
⑥ 画面	webapp/WEB-INF/jsp/oo.jsp	表示専用。受け取ったデータを表示するだけ	表示のことだけ

ポイント

- 画面 (JSP) に複雑な処理を書かない → 処理は全部 Logic に書く
- Logicに「SQLの書き方」を書かない → データの取得・更新は全部 DAO に任せる
- DAOは「DBにどう繋ぐか」だけを `dao/DBUtil.java` に集約する → 接続情報 (URL・ユーザー名・パスワード) が変わってもDBUtilだけ直せばよい
- それぞれの役割が分かれているので、「画面のデザインを直したい人」「処理のルールを直したい人」「データの中身 (テーブル) を直したい人」が、お互いのコードを壊さずに作業できる

2. 画面一覧・画面遷移表

すべての画面は `/ControlServlet?action=oo` という1つの入口から始まります。「どの `action` が来たら、どの `Logic`（または `ControlServlet` 自身）が処理して、どの画面（JSP）を表示するか」を一覧にしたものが下表です。新しい画面を追加するときは、この表に1行追加するイメージで `ControlServlet` に `case` を増やします。

action	担当	表示されるJSP	画面の役割
<code>login / doLogin</code>	<code>LoginLogic</code>	<code>login.jsp / loginNG.jsp</code>	ログイン画面・ログイン処理
<code>logout</code>	<code>LogoutLogic</code>	<code>logout.jsp</code>	ログアウト
<code>menu</code>	<code>ControlServlet#showMenu</code>	<code>menu.jsp</code>	一般ユーザーのメニュー（本日の予定・部署の出社状況）
<code>adminMenu</code>	<code>ControlServlet#showMenu</code>	<code>adminMenu.jsp</code>	管理者メニュー（一般メニュー+2週間分の予約サマリ）
<code>locationRegist / doLocationRegist</code>	<code>LocationRegistLogic</code>	<code>locationRegist.jsp</code> ほか	行先登録（出社・在宅・出張の振り分け）
<code>businessTrip / doBusinessTrip</code>	<code>LocationRegistLogic</code>	<code>businessTrip.jsp</code>	出張の登録（メモ入力）
<code>remoteWork / doRemoteWork</code>	<code>LocationRegistLogic</code>	<code>remoteWork.jsp</code>	在宅の登録（メモ入力）
<code>reserve / doReserve</code>	<code>ReserveLogic</code>	<code>reserve.jsp / reserveOK.jsp / reserveNG.jsp</code>	座席予約（拠点→フロア→エリア→座席の4段階）
<code>reserveConfirm / doCancel</code>	<code>ReserveConfirmLogic</code>	<code>reserveConfirm.jsp</code>	予約確認・キャンセル
<code>nameSearch / doNameSearch</code>	<code>NameSearchLogic</code>	<code>nameSearch.jsp</code>	氏名検索（社員の本日の状況を調べる）
<code>departmentSearch / doDepartmentSearch</code>	<code>DepartmentSearchLogic</code>	<code>departmentSearch.jsp</code>	部門検索（部署単位で本日の状況を調べる）
<code>seatStatus / doSeatStatus</code>	<code>ControlServlet#showSeatStatus</code>	<code>seatStatus.jsp</code>	座席利用状況（管理者）。拠点・フロア・エリア・日付で絞り込み
<code>adminReserve / doAdminReserve</code>	<code>AdminReserveLogic</code>	<code>adminReserve.jsp</code>	代理予約（管理者が他の人の予約を登録）
<code>masterUpdate / masterEdit / doMasterAdd / doMasterUpdate / doMasterDelete</code>	<code>MasterUpdateLogic</code>	<code>masterUpdate.jsp</code>	座席マスタの追加・更新・削除（管理者）
上記以外（未ログイン時など）	-	<code>login.jsp</code>	デフォルトはログイン画面

ログイン状態によるアクセス制限

`ControlServlet#process` の入口で、次のチェックを行っています（`login / doLogin` 以外の全アクションが対象）。

状態	結果
セッションに <code>userId</code> がある（ログイン済み）	そのまま処理を続ける
セッションが無い、または <code>userId</code> が無い	<code>action=login</code> にリダイレクト

管理者専用画面（`adminMenu`・`seatStatus`・`adminReserve`・`masterUpdate` など）かどうかの判定は、各JSP・Logic側でセッションの `isAdmin` を見て出し分けています（メニューのリンク表示や、ヘッダーの配色切り替えなど）。

3. フォルダ構成とファイル一覧

```
src/main/java/
├─ entity/    ... データの「入れ物」（クラス）。テーブルの1行に対応するイメージ
├─ dao/       ... データを取得・登録・更新・削除する係（JDBCでMySQLに接続）
├─ model/     ... 画面ごとの処理担当（ビジネスロジック）
├─ control/   ... 全リクエストの窓口（フロントコントローラ）

src/main/webapp/
├─ WEB-INF/jsp/ ... 画面表示担当（JSP）
├─ css/        ... 見た目（スタイルシート）
├─ images/     ... アイコン・フロアマップ画像
```

entity（4ファイル）

ファイル	対応するテーブル	内容
Account.java	users	ユーザー情報（ID・パスワード・氏名・部署・管理者フラグ）
Location.java	departments	部署情報（部署ID・部署名）
Seat.java	seats	座席情報（座席ID・拠点・フロア・エリア・座席名）
Reservation.java	reservations	予約・行先登録情報

dao（5ファイル）

ファイル	担当テーブル	主なメソッド
DBUtil.java	-	MySQLへの接続（ <code>getConnection()</code> ）を1箇所にまとめる共通クラス
AccountsDAO.java	users	<code>findById</code> <code>authenticate</code> <code>findAll</code> <code>findByName</code> <code>search</code> <code>findById</code>
LocationsDAO.java	departments	<code>findAll</code> <code>getBNameById</code> など部署名の取得
SeatsDAO.java	seats	<code>getAllSeats</code> <code>findById</code> <code>filterSeats</code> <code>getBases/getFloors/getAreas</code> <code>getOfficeImage</code> など
ReservationsDAO.java	reservations	<code>findTodayByUserId</code> <code>findByDate</code> <code>getSeatUsageForDate</code> <code>add</code> <code>cancel</code> など
MastersDAO.java	seats	マスタ更新画面専用の <code>getAll</code> <code>exists</code> <code>add</code> <code>update</code> <code>delete</code>

model（9ファイル）

ファイル	担当画面
Logic.java	全Logicクラスが実装するインターフェース（約束事）
LoginLogic.java	ログイン
LogoutLogic.java	ログアウト
LocationRegistLogic.java	行先登録（出社・在宅・出張）
ReserveLogic.java	座席予約
ReserveConfirmLogic.java	予約確認・キャンセル
NameSearchLogic.java	氏名検索
DepartmentSearchLogic.java	部門検索
AdminReserveLogic.java	代理予約
MasterUpdateLogic.java	座席マスタ更新

control（1ファイル）

ファイル	役割
ControlServlet.java	全リクエストの入口。 <code>action</code> の振り分けに加えて、メニュー画面（ <code>showMenu</code> ）と座席利用状況画面（ <code>showSeatStatus</code> ）の集計処理も自分でやっている

「直したいもの」から逆引き

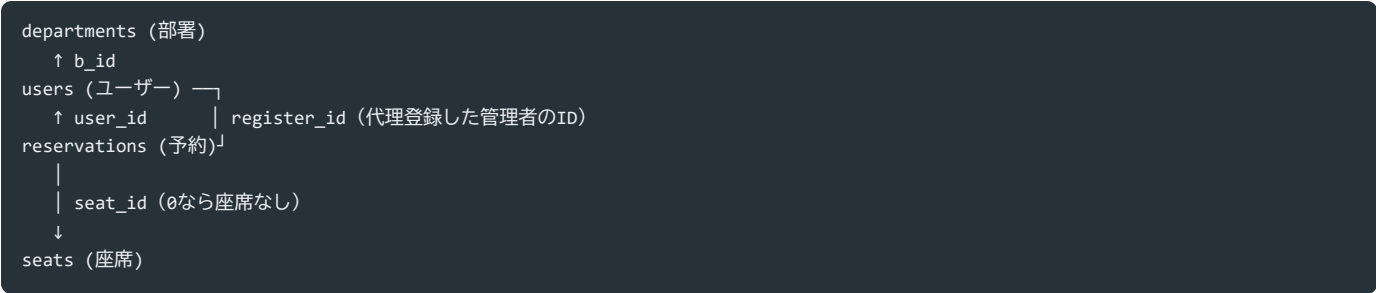
やりたいこと	触るべきフォルダ・ファイル
ボタンの色や文字サイズなど見た目を変えたい	<code>webapp/css/style.css</code>
画面に表示する項目・文言を変えたい	<code>webapp/WEB-INF/jsp/oo.jsp</code>
アイコン・フロアマップ画像を差し替えたい	<code>webapp/images/oo.png</code> （同じファイル名で上書きすればJSP側の修正は不要）
ヘッダーのメニュー項目を増やしたい・減らしたい	<code>webapp/WEB-INF/jsp/common/header.jspf</code> （一般） / <code>headerAdmin.jspf</code> （管理者）
「予約できる条件」など処理のルールを変えたい	<code>model/ooLogic.java</code>
表示するデータの内容（座席名・人数など）を変えたい	<code>dao/ooDAO.java</code> のSQL、または DBの中身（テーブル）
ユーザー・座席・部署のデータそのものを増減したい	MySQL（ <code>users</code> / <code>seats</code> / <code>departments</code> テーブル）を編集
DB接続情報（接続先・パスワード）を変えたい	<code>dao/DBUtil.java</code>
新しい画面・新しい機能を追加したい	<code>entity</code> → <code>dao</code> → <code>model</code> → <code>control</code> → <code>jsp</code> の順に作る

4. データベース構成（MySQL `zaseki1`）

座・NaviはMySQLのデータベース `zaseki1` を使います。テーブルは4つだけです。（定義一式は `docs/db/za_navi_schema.sql` にあります）

テーブル	役割	主なカラム
<code>departments</code>	部署マスタ	<code>b_id</code> （部署ID） / <code>b_name</code> （部署名）
<code>users</code>	アカウント	<code>user_id</code> （社員ID・数値） / <code>password</code> / <code>b_id</code> （所属部署） / <code>name</code> （氏名） / <code>admin</code> （1=管理者）
<code>seats</code>	座席マスタ	<code>seat_id</code> / <code>base_name</code> （拠点） / <code>f_name</code> （フロア） / <code>area_name</code> （エリア） / <code>seat_name</code> （座席名）
<code>reservations</code>	予約・行先登録	<code>reserve_id</code> / <code>user_id</code> （本人） / <code>register_id</code> （登録した人） / <code>reserve_date</code> / <code>place</code> （行先=出社/在宅/出張） / <code>seat_id</code> （座席なしは0） / <code>memo</code> / <code>cancel_flag</code> （1=取消済み）

テーブルの関係



覚えておきたいポイント

- `user_id` はDB上は **数値 (INT)** だが、画面・Java側では従来どおり **文字列 (String)** として扱う。 `AccountsDAO` の境界 (JDBCでデータを取り出す/渡すところ) だけで `Integer.valueOf(...)` ⇔ `String.valueOf(...)` の変換をしている。
- `reservations.seat_id = 0` は「座席なし」(在宅・出張、または座席未選択の出勤) を意味する。 `seats` テーブルへの外部キー制約は無い (0が存在しないため)。
- `reservations.cancel_flag = 1` は「キャンセル済み」。行は削除せず、フラグだけ立てる (履歴として残す)。
- 「同じ日・同じ座席に2人が予約できない」「1人が同じ日に2件の有効な予約を持ってない」は、 `seat_active_key` / `user_active_key` という生成列 + UNIQUE制約でDB側でも防いでいる (詳細は `za_navi_schema.sql` のコメント参照)。

5. 各レイヤーの詳しい説明

5-1. entity (エンティティ) = データの入れ物

`entity/Account.java`、`entity/Reservation.java`、`entity/Seat.java`、`entity/Location.java` の4つがあります。

```
public class Account {
    private String userId;    // ユーザーID (DB上はINTだが、画面側ではString)
    private String password;  // パスワード
    private String name;     // 氏名
    private String bId;      // 部門ID
    private int admin;       // 管理者フラグ (1=管理者, 0=一般)
    // ... getter/setter ...
}
```

これは「データベースの1行を表すクラス」です。例えば `users` テーブルの1行 (1人のユーザー情報) を、Javaのプログラムの中で扱いやすくするための「箱」だと考えてください。

ここを直すとき: 新しい項目 (例: 電話番号) を増やしたいときは、①DBのテーブルにカラムを追加 → ②entityにフィールドを追加 → ③DAOのSQL (SELECT/INSERTなど) に追加 → ④使う場所 (Logic・JSP) でも追加、という流れになります。

5-2. dao (データアクセスオブジェクト) = データの取得・更新係

`dao/AccountsDAO.java` を例に見てみましょう。

```

public class AccountsDAO {

    private Account map(ResultSet rs) throws SQLException {
        return new Account(
            String.valueOf(rs.getInt("user_id")),
            rs.getString("password"),
            rs.getString("name"),
            rs.getString("b_id"),
            rs.getInt("admin")
        );
    }

    public Account findById(String userId) {
        Integer id = parseUserId(userId);
        if (id == null) return null;

        String sql = "SELECT user_id, password, b_id, name, admin FROM users WHERE user_id = ?";
        try (Connection conn = DBUtil.getConnection();
            PreparedStatement ps = conn.prepareStatement(sql)) {
            ps.setInt(1, id);
            try (ResultSet rs = ps.executeQuery()) {
                if (rs.next()) return map(rs);
            }
        } catch (SQLException e) {
            throw new RuntimeException("アカウントの取得に失敗しました", e);
        }
        return null;
    }
}

```

ほぼすべてのDAOメソッドが、この形（**JDBCの基本パターン**）です。

手順	コード	やっていること
① 接続 を取得	<code>Connection conn = DBUtil.getConnection()</code>	MySQLにつなぐ
② SQL を準備	<code>PreparedStatement ps = conn.prepareStatement(sql)</code>	<code>?</code> のプレースホルダ付きSQLを用意する（SQLインジェクション対策にもなる）
③ 値を セット	<code>ps.setInt(1, id)</code> / <code>ps.setString(2, ...)</code>	<code>?</code> に実際の値を入れる
④ 実行	<code>ps.executeQuery()</code> （SELECT） / <code>ps.executeUpdate()</code> （INSERT/UPDATE/DELETE）	SQLを実行する
⑤ 結果 を変換	<code>map(rs)</code>	<code>ResultSet</code> （DBから返ってきた表）を <code>Account</code> などのentityに変換する
⑥ 後始 末	<code>try (...) { }</code>	<code>try-with-resources</code> で <code>Connection</code> / <code>PreparedStatement</code> / <code>ResultSet</code> を自動的にクローズする
⑦ 例外 処理	<code>catch (SQLException e) { throw new RuntimeException(...) }</code>	DBエラーをそのまま投げ直し、 <code>ControlServlet</code> 側でまとめてエラー画面に出す

- 旧バージョンでは `Arrays.asList(...)` という**Java内のモックデータ**を使っていましたが、現在は全DAOがこの形でMySQLに直接アクセスします。
- 「データを増やしたい・変えたい」場合は、Javaコードを直すのではなく **MySQLのテーブルの中身を直す**のが基本になりました（詳細は後述のQ&A参照）。

ここを直すとき: 表示するデータの取り方・絞り込み条件を変えたいときは、このSQL文（`String sql = "..."` の部分）を編集します。

5-3. model（ロジック）＝画面ごとの処理担当

`model/Logic.java` というインターフェース（約束事）を、各画面の処理クラスが実装しています。

```
public interface Logic {
    String execute(HttpServletRequest req, HttpServletResponse res) throws Exception;
}
```

「execute」という名前のメソッドを必ず作ってね、戻り値は次に表示するJSPのパス（文字列）にしてね」という約束です。

例えば LoginLogic.java:

```
public class LoginLogic implements Logic {
    @Override
    public String execute(HttpServletRequest req, HttpServletResponse res) {
        String userId = req.getParameter("userId");
        String password = req.getParameter("password");

        Account account = new AccountsDAO().authenticate(userId, password);

        if (account != null) {
            // ログイン成功 → セッションに情報を保存して、メニュー画面へ
            HttpSession session = req.getSession();
            session.setAttribute("userId", account.getUserId());
            session.setAttribute("userName", account.getName());
            session.setAttribute("isAdmin", account.getAdmin() == 1);
            return account.getAdmin() == 1
                ? "/WEB-INF/jsp/adminMenu.jsp"
                : "/WEB-INF/jsp/menu.jsp";
        } else {
            // ログイン失敗 → エラーメッセージを画面にセットして、ログイン画面へ
            req.setAttribute("errorMsg", "ユーザーIDまたはパスワードが正しくありません。");
            return "/WEB-INF/jsp/loginNG.jsp";
        }
    }
}
```

やっていることは3ステップです。

1. 画面から送られてきた値を受け取る（`req.getParameter(...)`）
2. DAOからデータを取得して、判定・加工する（業務のルール）
3. 画面に渡すデータをセットして（`req.setAttribute(...)`）、次に表示するJSPのパスを返す

⚠ 「ログインできません」と出たときの考え方: `AccountsDAO.authenticate()` が `null` を返すのは、「SQLは正常に実行できたが、その `user_id + password` に一致する行が無かった」という意味です。DB接続自体に失敗した場合は `SQLException` → `RuntimeException` となり、`ControlServlet` 側で「システムエラー」として別扱いになります。「ID/パスワードが違います」というメッセージが出ている時点で、DB接続自体は成功している、という切り分けができます。

ここを直すとき:「パスワードが3回間違ったらロックする」「土日は予約できないようにする」など、業務のルール・条件を変えたいときはここを編集します。

5-4. control（コントローラ）＝全リクエストの窓口

`control/ControlServlet.java` がアプリの「玄関」です。すべてのリクエストは `/ControlServlet?action=oo` という形でここに届きます。

```

switch (action) {
    case "login":
    case "doLogin":
        return new LoginLogic().execute(req, res);

    case "reserve":
    case "doReserve":
        return new ReserveLogic().execute(req, res);

    case "menu":
        return showMenu(req, false);

    case "adminMenu":
        return showMenu(req, true);
    // ...
}

```

「`action=login` というリクエストが来たら `LoginLogic` に処理を任せる」という振り分け表になっています（全パターンは「2. 画面一覧・画面遷移表」を参照）。

`menu` / `adminMenu` / `seatStatus` / `doSeatStatus` の4つだけは、専用の `Logic` クラスを作らず `ControlServlet` 自身のメソッド（`showMenu` / `showSeatStatus`）で処理しています（メニュー画面・座席利用状況画面は複数のDAOの結果を組み合わせる集計処理が多いため）。

ここを直すとき、新しい画面・機能を追加したときは、ここに `case` を1行追加して、対応する `Logic` を呼び出すようにします。

5-5. jsp（画面） = 表示担当

`webapp/WEB-INF/jsp/menu.jsp` などが画面の見た目を作っています。

```

<%@ page contentType="text/html; charset=UTF-8" pageEncoding="UTF-8" %>
...
<a href="<%= request.getContextPath() %>/ControlServlet?action=reserve" class="menu-btn">
    座席予約
</a>

```

- `<%@ page %>` は「このページの設定」を書く場所（文字コードなど）
- `<a href="...">` のリンクが押されると、また `ControlServlet` にリクエストが飛ぶ
- `<%= ... %>` の部分には、`Logic`が用意したデータ（`req.setAttribute(...)`したもの）を表示できます

ここを直すとき、文言・レイアウト・表示する項目を変えたいときはここを編集します。ロジック（計算や判定）をJSPに書かないのがルールです。

6. 「画面が動くまで」の具体例（ログイン画面で確認）

1. ブラウザで `http://localhost:8080/za-navi/ControlServlet?action=login` を開く
2. `ControlServlet` が `action=login` を受け取り、`LoginLogic` を呼ぶ
3. `LoginLogic` は何もせず（GETなので）`/WEB-INF/jsp/login.jsp` を返す
4. `login.jsp` が表示される
5. ユーザーがID・パスワードを入力して送信 → 今度は `action=doLogin` というリクエストが飛ぶ
6. `ControlServlet` が `LoginLogic` を呼ぶ（同じ`Logic`がログイン処理も担当）
7. `LoginLogic` が `AccountsDAO.authenticate(userId, password)` を呼び、内部で `SELECT ... FROM users WHERE user_id = ? AND password = ?` をMySQLに投げる
8. 一致する行があれば `Account` を返す。成功なら `menu.jsp`（または `adminMenu.jsp`）、失敗（該当行なし）なら `loginNG.jsp` を返す
9. `ControlServlet` がそのJSPに画面表示を依頼する

この「URLのaction → Logic → DAO（→ MySQL） → 結果に応じてJSPを選ぶ」という流れは、**すべての画面で共通**です。1つの画面の流れが理解できれば、他の画面も同じ読み方で理解できます。

7. 機能解説：実際のコードを読んでもみる

ここでは、実際にこのアプリに入っている「ちょっと工夫した処理」を取り上げて、コードがどう書いてあるか・どこを直せば動きが変わるかを具体的に見ていきます。

7-1. ヘッダーのプルダウンメニュー（JavaScriptを使わない実装）

`webapp/WEB-INF/jsp/common/header.jspf` を開くと、メニューが次のように書かれています。

```
<details class="menu-dropdown">
  <summary>
     <%= session.getAttribute("userName") %> さん <span class="caret">></span>
  </summary>
  <div class="dropdown-panel">
    <a href="...?action=menu">メニュー</a>
    <a href="...?action=logout">ログアウト</a>
  </div>
</details>
```

`<details>` と `<summary>` はHTML標準のタグで、JavaScriptを書かなくてもクリックで開閉するメニューが作れます。`<summary>` がクリックする部分（見出し）、その中の `<div class="dropdown-panel">` が開いたときに表示される中身です。見た目は `webapp/css/style.css` の `.menu-dropdown` `.dropdown-panel` で調整しています。

ここを直すとき：メニューの項目を増やしたい・並び替えたいときは、`<div class="dropdown-panel">` の中の `<a>` タグを編集します（一般ユーザー用は `header.jspf`、管理者用は `headerAdmin.jspf`）。

7-2. 行先登録で「出社」を選んだら座席予約画面へ進む

`model/LocationRegistLogic.java` の `doRegist` メソッドを見ると、行き先によって処理を分けています。

```
private String doRegist(HttpServletRequest req, HttpServletResponse res) throws Exception {
    String gyosaki = req.getParameter("gyosaki"); // 出社 / 在宅 / 出張
    if ("出張".equals(gyosaki)) {
        return "/WEB-INF/jsp/businessTrip.jsp";
    }
    if ("在宅".equals(gyosaki)) {
        return "/WEB-INF/jsp/remoteWork.jsp";
    }
    // 出社：ここでは登録せず座席予約画面（ReserveLogic）へリダイレクトし、
    // 座席が決まった時点で「出社+座席」をまとめて1件登録する
    // （そうしないと「出社（座席なし）」と「出社（座席あり）」の2件ができてしまうため）
    res.sendRedirect(req.getContextPath() + "/ControlServlet?action=reserve");
    return null;
}
```

ポイントは `res.sendRedirect(...)` と `return null` の組み合わせです。Logic インターフェース（`model/Logic.java`）の説明にある通り、`execute`（や内部メソッド）が `null` を返すのは「もう自分でリダイレクトしたので、`ControlServlet` は何もしなくてよい」という合図です。`return "/WEB-INF/jsp/xxx.jsp"` で**画面の切り替え（forward）をするのか、`res.sendRedirect(...)` + `return null` で別のactionへの飛び直し（redirect）**をするのか、の使い分けがここで学べます。

つまづきポイント: 以前の版では「出社」を選んだ時点でこの `doRegist` 内でも1件登録していたため、座席予約画面で座席を選んだときの登録と合わせて「出社（座席なし）」「出社（座席あり）」の2件が重複登録されてしまうバグがありました。「出社」のときは登録せずリダイレクトだけする＝登録は必ず1箇所（`ReserveLogic`）に集約する、という設計が直した後の形です。（現在はDB側でも `reservations` テーブルの `user_active_key` のUNIQUE制約により、1人が同じ日に有効な予約を複数件持つことは構造上できないようになっています。）

「在宅」「出張」も専用画面でメモを書いてから登録: どちらも一旦 `remoteWork.jsp` / `businessTrip.jsp` に forward し、利用者が任意でメモを入力してから `doRemoteWork` / `doBusinessTrip` で登録する流れになっています。登録後は `saveAndGoMenu` が `ControlServlet?action=menu`（または管理者なら `adminMenu`）へ `redirect` し、セッションの `isAdmin` を見てメニューを振り分けます（完了メッセージはリダイレクトをまたいで表示する必要があるため、リクエスト属性ではなくセッションの一時的な「フラッシュメッセージ」として持たせています）。

つまずきポイント: LocationRegistLogic 側に新しい action (remoteWork / doRemoteWork) の処理を追加しても、入口の ControlServlet の switch に対応する case を足し忘れると、default 分岐でログイン画面に飛ばされてしまいます。**新しい action を追加するときは、必ず ControlServlet と各 Logic クラスの両方に手を入れる、**という基本を再確認できる典型例です。

ここを直すとき: 行き先ごとの遷移ルールを変えたいときはこの if 分岐を、登録後の戻り先や完了メッセージを変えたいときは saveAndGoMenu を編集します。

7-3. 検索結果に座席情報を追加する (DAOを組み合わせる例)

model/NameSearchLogic.java ・ DepartmentSearchLogic.java では、検索結果の各ユーザーについて「本日の予約」から「予約している座席」まで芋づる式に取得しています。

```
List<Reservation> todayRes = resDAO.findTodayByUserId(a.getUserId());
String todayStatus = "未登録";
Seat seat = null;
if (!todayRes.isEmpty()) {
    Reservation r = todayRes.get(0);
    todayStatus = r.getGyosaki();
    if (r.getSeatId() > 0) {
        seat = seatDAO.findById(r.getSeatId()); // 予約 → 座席IDから座席情報を取得
    }
}
row.put("todayStatus", todayStatus);
row.put("seat", seat); // JSPへ渡す
```

ReservationsDAO で「その人が今日どこに行く予定か」を調べ、座席を予約していれば seatId を使って SeatsDAO から座席の詳細 (拠点・フロア・エリア・座席番号) を取得する、という**2段階の問い合わせ**になっています。JSP側 (nameSearch.jsp ・ departmentSearch.jsp) では、受け取った seat が null でなければ seat.getBaseName() などを表示するだけです。

ここを直すとき: 検索結果に表示する項目をさらに増やしたい場合は、同じ要領で「Logicで必要なデータをDAOから集めて row に詰める」→「JSPで row.get(...) して表示する」という形を真似すれば追加できます。

7-4. 拠点ごとのフロアマップ画像を切り替える仕組み (OFFICE_IMAGES)

座席利用状況・代理予約・マスタ更新・座席予約の各画面では、「拠点 (オフィス) 」や「フロア」「エリア」を選ぶとそれに対応するマップ画像が表示されます。この対応関係は dao/SeatsDAO.java の OFFICE_IMAGES という Map にまとめられています。

```
static final Map<String, String> OFFICE_IMAGES = new LinkedHashMap<>();
static {
    OFFICE_IMAGES.put("三田_1F", "office_mita_1F.png");
    OFFICE_IMAGES.put("芝浦_1F", "office_shibaura_1F.png");
    // 拠点単位のフォールバック画像
    OFFICE_IMAGES.put("三田", "office_mita.png");
    // エリア専用画像
    OFFICE_IMAGES.put("三田_1F_A", "office_mita_1F_A.png");
    // ...
}

public String getOfficeImage(String baseName, String floorName) {
    if (floorName != null && !floorName.isEmpty()) {
        String perFloor = OFFICE_IMAGES.get(baseName + "_" + floorName);
        if (perFloor != null) return perFloor;
    }
    return OFFICE_IMAGES.get(baseName);
}
```

キーの付け方には次のルールがあります。

キーの形	例	使われる場面
拠点名	"三田"	フロア未選択時のフォールバック画像
拠点名_フロア名	"三田_1F"	フロアまで選んだときに優先表示される画像
拠点名_フロア名_エリア名	"三田_1F_A"	エリアまで選んだときの専用画像（座席予約画面など）

`getExpectedFloorImage` / `getExpectedAreaImage` は、`OFFICE_IMAGES` に未登録の拠点・フロア・エリアでも「こういうファイル名のはず」という期待されるファイル名を自動生成して返します。JSP側は常にこのファイル名で `<img>` タグを出し、画像ファイルが実際に無い場合は `onerror` でプレースホルダー表示に切り替えます。これにより、新しい拠点・フロアを追加した直後でも画面が壊れず、後から画像ファイルを置けば自動的に表示されるようになっています。

新しいオフィスを増やす手順（`SeatsDAO.java` の冒頭コメントにも同じ内容を記載しています）：

1. `seats` テーブルに新オフィス分の座席データを追加する（マスタ更新画面の「②座席を追加する」から「+ 新しいオフィスを追加する」を選んで可）
2. フロアマップ画像（PNGなど）を用意し、`webapp/images/` と Eclipse側 `WebContent/images/` の両方に置く
3. 必要であれば `OFFICE_IMAGES` に "拠点名_フロア名" などのキーで対応を追記する（追記しなくても、上記の自動生成ファイル名で画像を置けば表示される）

ここを直すとき: オフィスを追加・削除したいときはこのセクションの手順どおりに `seats` テーブルと `SeatsDAO.java` を編集します。画像を変えたいただけなら、同じファイル名で画像を上書きするだけでOK（コードの変更は不要）。

7-5. 「拠点 → フロア → エリア」を順番に絞り込む仕組み（カスケード選択）

座席利用状況画面・代理予約画面・座席予約画面では、「①拠点を選ぶ → ②その拠点にあるフロアだけが選べるようになる → ③そのフロアにあるエリアだけが選べるようになる」という、前の選択に応じて次の選択肢が変わるカスケード選択を採用しています。「すべての拠点」のような曖昧な状態を作らず、常に「〇〇オフィスの〇〇エリア」のように場所が一意に決まるようにするためです。

仕組みは難しくありません。`ControlServlet#showSeatStatus` などで、選ばれている値に応じて段階的にリストを用意するだけです。

```
boolean baseChosen = filterBase != null && !filterBase.isEmpty();
boolean floorChosen = baseChosen && filterFloor != null && !filterFloor.isEmpty();
boolean areaChosen = floorChosen && filterArea != null && !filterArea.isEmpty();

if (baseChosen) req.setAttribute("floors", seatsDAO.getFloors(filterBase));
if (floorChosen) req.setAttribute("areas", seatsDAO.getAreas(filterBase, filterFloor));

List<Seat> filteredSeats = areaChosen
    ? seatsDAO.filterSeats(filterBase, filterFloor, filterArea)
    : new ArrayList<>();
```

「拠点が選ばれていなければフロアの選択肢は用意しない」「フロアが選ばれていなければエリアの選択肢は用意しない」という順番がポイントです。`getFloors` / `getAreas` は、内部で `SELECT DISTINCT f_name FROM seats WHERE base_name = ?` のようなSQLを実行し、そのときDBに実際に存在する値だけを選択肢として返します。JSP側は `<select onchange="this.form.submit()">` でプルダウンを変更するたびにフォームを自動送信し、`floors` / `areas` が `null` の間は `<select disabled>` にして選ばないようにしています。

```
<select name="filterFloor" onchange="this.form.submit()" <%= (floors == null) ? "disabled" : "" %>>
...
</select>
```

ここを直すとき: 同じ「親を選ぶまで子は選べない」絞り込みを別の画面でも作りたいときは、Logic（またはControlServlet）側で「選ばれているか」のbooleanを段階的に作り、それに応じてリストをセットする → JSP側は `disabled` とプルダウンの自動送信、という形を真似します。

7-6. 管理者ログイン中だけ画面の色を変える仕組み（CSS変数の上書き）

管理者でログインしている間は、ヘッダーやボタンの基調色がティール系からインディゴ系に変わります。`webapp/css/style.css` の `:root` で色はCSS変数として定義されており（`--teal` `--teal-light` `--teal-dark`）、ボタンやヘッダーなどあらゆる場所がこの変数を参照しています。

```
:root {
  --teal:      #00897B;
  --teal-light: #4DB6AC;
  --teal-dark:  #00695C;
}
.header { background: var(--teal); }
.btn-primary { background: var(--teal); color: #fff; }
```

管理者用ヘッダー（`headerAdmin.jspf`）と一般用ヘッダー（`header.jspf`）の先頭で、ログイン中のユーザーが管理者なら、この変数を上書きする `<style>` を出力しています。

```
<% if (Boolean.TRUE.equals(session.getAttribute("isAdmin"))) { %>
<style>
  :root { --teal:#3949AB; --teal-light:#7986CB; --teal-dark:#283593; }
</style>
<% } %>
```

CSSの `:root` はどこに `<style>` を書いても文書全体に効くため、ヘッダーの中に書くだけでページ全体の色が一括で変わります。個別の `.btn` や `.header` を1つずつ書き換える必要がないのが、CSS変数を使う一番のメリットです。

ここを直すとき: 配色そのものを変えたいときはこの `<style>` 内の3つの色コードを変更するだけ。「管理者だけでなく特定の部署だけ色を変えたい」など条件を変えたい場合は、`if` の条件式を書き換えます。

7-7. メニュー画面の集計表示（同部署の出勤人数・2週間分の予約状況）

メニュー画面の表示は `control/ControlServlet.java` の `showMenu` メソッドが担当しています（このアプリでは `MenuLogic` という専用クラスを作らず、コントローラの中で直接処理しています）。

```
// 同じ部署の本日の出勤人数・座席一覧
Account me = accDAO.findById(userId);
List<Account> deptMembers = accDAO.findByBId(me.getBId());

List<Map<String, Object>> deptSeatList = new ArrayList<>();
long officeCount = 0;
for (Account a : deptMembers) {
  List<Reservation> mRes = resDAO.findTodayByUserId(a.getUserId());
  if (mRes.isEmpty()) continue;
  Reservation r = mRes.get(0);
  if (!"出勤".equals(r.getGyosaki())) continue;
  officeCount++;

  Seat seat = r.getSeatId() > 0 ? seatDAO.findById(r.getSeatId()) : null;
  Map<String, Object> row = new HashMap<>();
  row.put("name", a.getName());
  row.put("seat", seat);
  deptSeatList.add(row);
}
req.setAttribute("deptOfficeCount", officeCount);
req.setAttribute("deptSeatList", deptSeatList);
```

「自分の部署のメンバー一覧を取得 → 一人ひとりの本日の予約を調べる → 出勤 の人だけ人数を数えつつ、氏名と座席をリストに詰める」という流れです。人数だけでなく「誰が・どの座席にいるか」までその場で組み立てているのがポイントで、JSP側（`menu.jsp` / `adminMenu.jsp`）はこの `deptSeatList` を `for` で回して `row.get("name")` ・ `row.get("seat")` を表示するだけになっています。

2週間分の予約状況（管理者メニューのみ表示）も同じ考え方で、`LocalDate.now().plusDays(i)` で14日分の日付を作りながら、1日ごとに `findByDate` で予約を取得し、件数の集計と「氏名＋座席」の明細リスト（`people`）の両方を組み立てています。

```

for (int i = 0; i < 14; i++) {
    String date = LocalDate.now().plusDays(i).toString();
    List<Reservation> dayRes = resDAO.findByDate(date);
    long officeNum = dayRes.stream().filter(r -> "出社".equals(r.getGyosaki())).count();
    long seatNum = dayRes.stream().filter(r -> r.getSeatId() > 0).count();

    // 出社・座席が確定している人の「氏名+座席」明細（折りたたみ表示用）
    List<Map<String, Object>> people = new ArrayList<>();
    for (Reservation r : dayRes) {
        if (!"出社".equals(r.getGyosaki()) || r.getSeatId() <= 0) continue;
        Account acc = accDAO.findById(r.getUserId());
        Seat seat = seatDAO.findById(r.getSeatId());
        people.add(...氏名と座席を詰めたMap...);
    }
    row.put("people", people);
    ...
}

```

adminMenu.jsp 側では、この people を `<details class="day-summary"><summary>...</summary><div class="day-detail">...</div></details>` という HTML 標準の折りたたみ要素で包み、「閉じた状態は日付と件数だけのコンパクト表示」「クリックして開くと氏名・座席まで見える詳細表示」を、JavaScript なしで実現しています（7-1のプルダウンメニューと同じ `<details> / <summary>` パターン）。

ここを直すとき：

- 件数の数え方や表示する範囲（2週間→1ヶ月など）を変えたい → showMenu メソッドの集計部分を編集
- 表示するレイアウトを変えたい → menu.jsp（一般） / adminMenu.jsp（管理者）の該当する `<div class="card">` を編集
- 一般ユーザーにも2週間分を見せたい → `if (isAdmin) { ... }` の条件を外し、menu.jsp 側にも同じ表示ブロックを追加する

7-8. マスタ更新の重複チェック（DAOで「既に存在するか」を調べる）

マスタ更新画面では、座席を追加・更新するときに「拠点・フロア・エリア・座席名」が完全に一致する座席が既にあれば、エラーメッセージを表示して登録を防ぎます。

```

// MastersDAO.java
public boolean exists(String baseName, String fName, String areaName, String seatName, int excludeSeatId) {
    String sql = "SELECT 1 FROM seats WHERE base_name=? AND f_name=? AND area_name=? AND seat_name=? AND seat_id<>?";
    try (Connection conn = DBUtil.getConnection();
        PreparedStatement ps = conn.prepareStatement(sql)) {
        ps.setString(1, baseName);
        ps.setString(2, fName);
        ps.setString(3, areaName);
        ps.setString(4, seatName);
        ps.setInt(5, excludeSeatId);
        try (ResultSet rs = ps.executeQuery()) {
            return rs.next(); // 1行でも見つければ true
        }
    } catch (SQLException e) {
        throw new RuntimeException("座席の重複チェックに失敗しました", e);
    }
}

```

excludeSeatId は「更新中の自分自身は重複チェックの対象から外す」ための仕組みです。

呼び出し元	excludeSeatId	意味
doMasterAdd（新規追加）	-1	全座席が対象（自分はまだ存在しないので除外不要）
doMasterUpdate（更新）	更新対象の seatId	「自分以外」に同じ内容の座席があれば重複エラー

ここを直すとき：「拠点名だけ重複NG」のように重複判定の基準を変えたい場合は、このSQLの WHERE 条件（AND で繋がっている列）を変更します。

8. よくある「ここを直したい」Q&A

Q. ボタンの表示文言を変えたい → 該当する `.jsp` ファイルを開いて、文字列を直接書き換えればOK。

Q. ログインできるユーザーを増やしたい → MySQLの `users` テーブルに行を1件追加します。例:

```
INSERT INTO users (user_id, password, b_id, name, admin)
VALUES (1201, 'pass1201', 'D001', '新入社員', 0);
```

(`user_id` は既存の値と重複しないようにすること。Javaコードの修正は不要です。)

Q. 座席を増やしたい・座席のレイアウトを変えたい → 画面から行う場合は「マスタ更新」画面（管理者）の「②座席を追加する」フォームを使うのが簡単です（重複チェック付き）。直接SQLで行う場合は `seats` テーブルに `INSERT` します。

Q. 部署を増やしたい → `departments` テーブルに `INSERT` してから、その `b_id` を持つユーザーを `users` に追加します。

Q. 「土日は予約できない」などのルールを追加したい → `model/ReserveLogic.java` の処理の中に、日付を判定する `if` 文を追加。

Q. 新しい画面を追加したい → 次の順番で作成します。

1. 必要なら `entity` にデータの入れ物を追加
2. `dao` にデータ取得用のメソッドを追加（SQLを書く）
3. `model` に処理クラス（`Logic` を実装）を作成
4. `control/ControlServlet.java` に `case` を追加（「2. 画面一覧・画面遷移表」に1行追加するイメージ）
5. `webapp/WEB-INF/jsp` に画面（JSP）を作成

Q. DBに繋がらない・エラーになる → まず `dao/DBUtil.java` の接続情報（URL・ユーザー名・パスワード）が自分のMySQL環境と合っているか確認してください。「ユーザーIDまたはパスワードが正しくありません」は接続自体は成功しているケースなので、別の原因（DBの中身を確認）です（「5-3. model」のコラム参照）。

関連ドキュメント:

- [02 DB接続ガイド.md](#) — モックデータから実際のMySQLに繋ぎ変えた手順・経緯
- [03 研修ハンズオン手順書.md](#) — 研修当日の手順まとめ
- [06 試験要領書.pdf](#) — テスト項目（T01～T12）一覧
- [07 不具合修正履歴書.pdf](#) — 開発中に見つかった不具合と対応の履歴
- [docs/db/za_navi_schema.sql](#) — データベースのテーブル定義・初期データ